Especificación de Requisitos de Software (SRS)

BlockBridge Academy v2 — Quizzes de programación con bloques

Norma de referencia	ISO/IEC/IEEE 29148:2018 (estructura compatible con IEEE 830-1998)
Versión	2.0
Fecha	29 de octubre de 2026
Autor	[Nombre del desarrollador]

---

1. Introducción

1.1 Propósito

Este documento especifica los requisitos funcionales y no funcionales del MVP de BlockBridge Academy v2. Está dirigido al desarrollador, al profesor evaluador de la materia y a cualquier persona que valide o amplíe el sistema.

1.2 Alcance del producto

BlockBridge Academy v2 es una aplicación web en la que los estudiantes resuelven quizzes de programación armando programas con bloques visuales. El sistema muestra el código equivalente en 8 lenguajes, valida las respuestas comparando la estructura de los bloques con las soluciones registradas, aplica una penalización por cada pista usada y califica de 0 a 100. Los docentes crean contenido y consultan resultados. El alcance completo, lo que queda fuera y lo excluido se detallan en el documento 00, sección 4.

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
Bloque	Pieza visual de Blockly que representa una instrucción o expresión
Workspace	Área de trabajo de Blockly; se serializa como JSON con `Blockly.serialization.workspaces.save`
Toolbox	Paleta de bloques disponibles para un ejercicio
Forma canónica	Representación normalizada de un workspace para compararlo (Anexo B)
Ejercicio	Enunciado + toolbox + soluciones aceptadas + pistas + explicación
Quiz	Conjunto ordenado de ejercicios con configuración (lenguajes, penalización por pista)
Sesión de quiz	Una vez que un usuario resuelve un quiz; guarda intentos, pistas, tiempos y puntaje

Término	Definición
ID token	JWT que emite Firebase Authentication y verifica el servidor
Security Rules	Reglas declarativas de Firestore que autorizan cada lectura y escritura del cliente
TTL	*Time-to-live*: política de Firestore que borra documentos al vencer un campo de fecha
ARCO	Derechos de Acceso, Rectificación, Cancelación y Oposición (LFPDPPP)
MVP	Producto mínimo viable
DBA	Administrador de base de datos
MoSCoW	Prioridad: **M**ust (debe), **S**hould (debería), **C**ould (podría), **W**on't (no en esta versión)

1.4 Referencias

1. ISO/IEC/IEEE 29148:2018 — *Systems and software engineering — Life cycle processes — Requirements engineering*.
2. IEEE 830-1998 — *Recommended Practice for Software Requirements Specifications*.
3. Documentación de Blockly — developers.google.com/blockly.
4. Documentación de Firebase Authentication, Cloud Firestore y Security Rules — firebase.google.com/docs.
5. Documentación de Vercel Functions — vercel.com/docs.
6. Ley Federal de Protección de Datos Personales en Posesión de los Particulares (México, DOF 20-03-2025).
7. WCAG 2.1 — *Web Content Accessibility Guidelines*, nivel AA.

1.5 Visión general del documento

La sección 2 describe el producto en términos generales. La sección 3 enumera los requisitos verificables. La sección 4 indica cómo se verificará cada grupo de requisitos. Los anexos contienen el catálogo de bloques, el algoritmo de comparación y la matriz de trazabilidad.

---

2. Descripción general

2.1 Perspectiva del producto

Es un sistema nuevo e independiente que sustituye a la v1 (ver doc. 00 §8). Se compone de un cliente web estático, funciones *serverless* en Vercel y servicios gestionados de Firebase (diagrama en el doc. 00 §7).

2.2 Funciones del producto

1. Gestión de identidad: uso anónimo, registro, inicio de sesión, vinculación y eliminación de la cuenta.
2. Resolución de quizzes con bloques y vista de código en 8 lenguajes.

3. Evaluación en el servidor: comparación estructural, pistas con penalización, tiempos y puntaje.
4. Gestión de contenido por parte del docente: ejercicios y quizzes.
5. Analítica docente: dashboard y exportación.
6. Cumplimiento de privacidad: avisos, consentimiento, expiración y supresión de datos.

2.3 Características de los usuarios

Actor	Perfil	Competencia técnica
Visitante / Estudiante	Estudiante de cualquier nivel, desde bachillerato hasta posgrado, que aprende a programar	Uso básico del navegador; sin conocimientos previos de programación
Docente	Profesor de programación	Domina al menos uno de los 8 lenguajes
Administrador / DBA	El propio desarrollador en el MVP	Maneja la Consola de Firebase

2.4 Restricciones

- **R-01** Un solo desarrollador.
- **R-02** Cliente en JavaScript nativo, sin framework y sin paso de compilación.
- **R-03** Hosting en Vercel (plan Hobby, uso no comercial) y Firebase en plan Spark (gratuito).
- **R-04** No se ejecuta código del estudiante.
- **R-05** El rol de un usuario solo se cambia desde la Consola de Firebase.

2.5 Supuestos y dependencias

Los supuestos S-01 a S-08 están en el documento 00 §5. Dependencias externas: disponibilidad de Firebase, de Vercel y del CDN que sirve Blockly y el SDK de Firebase.

2.6 Reglas de negocio

ID	Regla
RN-01	**Puntaje de un ejercicio:** `resuelto ? max(0, 100 − h × p) : 0`, donde `h` es el número de pistas usadas y `p` la penalización por pista del quiz
RN-02	**Puntaje de un quiz:** promedio aritmético de los puntajes de sus ejercicios, todos con el mismo peso, redondeado a 1 decimal. Los ejercicios no resueltos u omitidos valen 0
RN-03	Los intentos son ilimitados y no penalizan; solo se cuentan
RN-04	Cada ejercicio tiene de 0 a 3 pistas, que se revelan en orden. `p` se configura por quiz entre 0 y 50 (por defecto 20)
RN-05	Un ejercicio resuelto queda bloqueado en esa sesión: no admite más envíos ni pistas

ID	Regla
RN-06	El tiempo no afecta el puntaje. El inicio y el fin de la sesión usan la hora del servidor; el tiempo de cada ejercicio lo reporta el navegador (solo mientras el ejercicio está a la vista) y el servidor lo acota a la duración real de la sesión
RN-07	Una respuesta es correcta si su forma canónica es idéntica a la de alguna solución aceptada del ejercicio (Anexo B)
RN-08	El dashboard muestra el **mejor** puntaje de cada estudiante por quiz y el número de sesiones que hizo
RN-09	El puntaje de la sesión se actualiza con cada respuesta correcta. *Finalizar* solo cierra la sesión y detiene el cronómetro
RN-10	La POO no aplica a C: los ejercicios con bloques de POO excluyen C de sus lenguajes
RN-11	Un docente solo edita sus propios ejercicios y quizzes. Los precargados son de solo lectura, aunque se pueden duplicar
RN-12	Las sesiones de usuarios anónimos expiran a los 30 días
RN-13	Un ejercicio usado en alguna sesión no se borra: se archiva
RN-14	Todo usuario nuevo recibe el rol `estudiante`; ningún flujo de la aplicación puede cambiarlo

---

3. Requisitos específicos

3.1 Interfaces externas

ID	Interfaz	Descripción
IE-01	Interfaz de usuario	Páginas HTML responsive. Pantallas: Inicio/Catálogo, Registro/Login, Quiz (enunciado, editor, código, pistas, cronómetro, retroalimentación), Resumen, Historial, Docente (ejercicios, quizzes, dashboard), Aviso de privacidad, Mi cuenta
IE-02	Firebase Authentication	SDK modular: `signInAnonymously`, `createUserWithEmailAndPassword`, `linkWithCredential`, `signInWithEmailAndPassword`, `sendPasswordResetEmail`, `sendEmailVerification`, `getIdToken`
IE-03	Cloud Firestore	Colecciones `usuarios`, `ejercicios` (+ subcolección `privado`), `quizzes` y `sesiones` (modelo en el doc. 04)

ID	Interfaz	Descripción
IE-04	API propia (Vercel Functions, firma web `POST(request: Request): Response`)	JSON sobre HTTPS con encabezado `Authorization: Bearer <ID token>`: `POST /api/sesiones` iniciar sesión de quiz, o `{accion: "reclamar"}` para conservar las sesiones anónimas al registrarse `POST /api/pista` revelar la siguiente pista `POST /api/respuesta` enviar un workspace `POST /api/finalizar` cerrar la sesión `POST /api/ejercicios` guardar, duplicar, archivar o borrar un ejercicio (docente) `POST /api/quizzes` guardar o borrar un quiz (docente) `DELETE /api/cuenta` eliminar la cuenta y sus datos (exige contraseña escrita en los últimos 5 minutos) `GET /api/limpieza` tarea diaria de Vercel Cron que aplica los plazos de conservación (protegida con `CRON_SECRET`)
IE-05	Hardware	Ratón, teclado o pantalla táctil; ancho mínimo de 360 px

3.2 Requisitos funcionales

3.2.1 Identidad y cuentas

ID	Requisito	Prioridad
RF-01	El sistema debe permitir usar los quizzes sin registrarse, creando una sesión anónima de Firebase que no pida ningún dato personal	M
RF-02	El sistema debe permitir registrarse con apodo (3–30 caracteres), correo y contraseña. Antes de crear la cuenta exige aceptar el aviso de privacidad y declarar mayoría de edad o autorización del tutor	M
RF-03	Si un usuario anónimo se registra, el sistema debe vincular la credencial a la misma cuenta (`linkWithCredential`) para conservar sus sesiones, y quitarles la marca de expiración	M
RF-04	El sistema debe permitir iniciar sesión con correo y contraseña	M
RF-05	El sistema debe permitir recuperar la contraseña por correo, mostrando el mismo mensaje exista o no la cuenta	M
RF-06	El sistema debe permitir cerrar sesión	M
RF-07	Al registrarse, el sistema debe crear el perfil con `rol = "estudiante"`. Las reglas impiden que el cliente cree o modifique cualquier otro valor de `rol`	M
RF-08	El sistema solo habilita las funciones de docente si `rol = "docente"` **y** el correo está verificado	M
RF-09	El sistema debe permitir que un usuario registrado descargue sus datos en JSON y elimine su cuenta y todos sus datos personales, previa reautenticación (`DELETE /api/cuenta`)	M

ID	Requisito	Prioridad
RF-10	El sistema debe permitir que el usuario cambie su apodo	S

3.2.2 Resolución de quizzes

ID	Requisito	Prioridad
RF-11	El sistema debe mostrar un catálogo de quizzes publicados, filtrable por tema, dificultad y lenguaje	M
RF-12	El sistema debe permitir elegir uno de los lenguajes que el quiz habilita, entre Java, Kotlin, C#, C++, C, Python, JavaScript y PHP	M
RF-13	Al iniciar un quiz, el servidor debe crear una sesión con la hora de inicio y devolver solo los datos públicos de los ejercicios: enunciado, toolbox, workspace inicial, tema, dificultad y número de pistas disponibles	M
RF-14	El sistema debe mostrar un editor de bloques con la toolbox del ejercicio y, opcionalmente, un workspace inicial	M
RF-15	El sistema debe mostrar en vivo el código generado en el lenguaje elegido y permitir cambiar de lenguaje sin alterar los bloques	M
RF-16	El sistema debe mostrar un cronómetro por ejercicio y otro total	M
RF-17	El sistema debe revelar las pistas de una en una, en orden. Antes de revelarla muestra la penalización y el puntaje máximo restante y pide confirmación; el servidor registra el uso	M
RF-18	El sistema debe enviar el workspace al servidor para que lo valide estructuralmente (Anexo B). El cliente nunca recibe las soluciones	M
RF-19	Si la respuesta es correcta, el sistema debe mostrar de inmediato el puntaje obtenido y la explicación del ejercicio	M
RF-20	Si la respuesta es incorrecta, el sistema debe indicar de inmediato el tipo de diferencia (*bloque distinto*, *valor distinto*, *falta un bloque* o *sobra un bloque*) y resaltar en el editor el primer bloque donde se aparta de la solución más cercana	M
RF-21	El sistema debe aceptar intentos ilimitados sin penalización y contar cada uno	M
RF-22	El sistema debe permitir moverse entre ejercicios y omitir uno	M
RF-23	El sistema debe calcular los puntajes según RN-01 y RN-02	M
RF-24	Al finalizar, el sistema debe mostrar un resumen con el puntaje total y, por ejercicio, el estado, los intentos, las pistas, el tiempo y el puntaje	M
RF-25	El estudiante registrado debe poder consultar su historial de sesiones	S
RF-26	El sistema debería guardar el workspace en curso en el navegador para no perderlo al recargar la página	C

3.2.3 Gestión de contenido (Docente)

ID	Requisito	Prioridad
RF-27	El docente debe poder crear, editar, archivar y listar sus ejercicios. Datos: título, enunciado, tema, dificultad, toolbox, workspace inicial, de 1 a 5 soluciones armadas con bloques, de 0 a 3 pistas y explicación	M
RF-28	El sistema debe calcular automáticamente los lenguajes aplicables a un ejercicio: si usa bloques de POO, excluye C	M
RF-29	El docente debe poder probar un ejercicio en vista previa antes de publicarlo	S
RF-30	El docente debe poder crear, editar, publicar y despublicar quizzes. Datos: título, descripción, ejercicios ordenados, lenguajes permitidos (intersección de los aplicables) y penalización por pista	M
RF-31	El docente debe poder ver los ejercicios precargados y duplicarlos para editar la copia	S

3.2.4 Analítica (Docente)

ID	Requisito	Prioridad
RF-32	Para cada quiz del docente o precargado, el sistema debe mostrar un dashboard con: número de sesiones, promedio, distribución de puntajes en 5 rangos, y por ejercicio el % de acierto, los intentos, pistas y tiempo promedio. También una tabla por estudiante registrado con apodo, mejor puntaje, número de sesiones, tiempo y fecha	M
RF-33	Los resultados de usuarios anónimos solo se muestran agregados, nunca por persona	M
RF-34	El docente debe poder exportar la tabla del dashboard a CSV en UTF-8 con BOM, para que Excel muestre bien los acentos	M
RF-35	El docente debe poder exportar el dashboard a PDF con una hoja de estilos de impresión	M

3.2.5 Contenido precargado y privacidad

ID	Requisito	Prioridad
RF-36	El sistema debe incluir un banco precargado de al menos 27 ejercicios (3 por cada uno de los 9 temas, uno por nivel de dificultad) y al menos un quiz por tema, cargados con un script del DBA	M
RF-37	El sistema debe mostrar el aviso de privacidad simplificado en el registro y enlazar el integral desde el pie de todas las páginas	M
RF-38	El sistema debe registrar la versión y la fecha del aviso aceptado	M
RF-39	El sistema debe marcar con `expiraEn` las sesiones anónimas para que la limpieza diaria las borre a los 30 días	M

3.3 Requisitos de usabilidad y accesibilidad

ID	Requisito
RNF-USA-01	Diseño responsive de 360 px a 1920 px sin desplazamiento horizontal de la página. En pantallas menores de 768 px, el enunciado, el editor y el código se muestran en pestañas
RNF-USA-02	Los bloques se pueden arrastrar con el dedo en pantallas táctiles
RNF-USA-03	Contraste WCAG 2.1 AA (4.5:1 en texto) en la interfaz y en los colores de bloque
RNF-USA-04	Todos los formularios, botones y menús son operables con teclado y tienen foco visible. La navegación por teclado *dentro* del lienzo de bloques queda fuera del MVP
RNF-USA-05	Interfaz en español; los mensajes de error explican qué pasó y qué hacer
RNF-USA-06	Un estudiante nuevo completa su primer ejercicio sin instrucciones externas (prueba con 3 usuarios)

3.4 Requisitos de rendimiento

ID	Requisito
RNF-REN-01	50 usuarios simultáneos sin degradación perceptible
RNF-REN-02	Carga inicial de la página de quiz < 3 s en una conexión 4G simulada, con caché fría
RNF-REN-03	Respuesta de `/api/respuesta` < 1 s en el p95 con la función ya en caliente; < 3 s en arranque en frío
RNF-REN-04	Regeneración del código al mover un bloque < 100 ms
RNF-REN-05	El dashboard de un quiz con 500 sesiones carga en < 3 s

3.5 Requisitos lógicos de la base de datos

El modelo de documentos de Firestore, el diagrama entidad-relación y la matriz de permisos están en el **documento 04**. Las restricciones de integridad que no puede imponer Firestore se validan en las Security Rules (tipos, campos permitidos, propietario) o en las funciones de Vercel.

3.6 Requisitos de seguridad

ID	Requisito
RNF-SEG-01	Las Security Rules niegan todo por defecto y solo permiten lo que enumera la matriz de permisos (doc. 04 §5)

ID	Requisito
RNF-SEG-02	**Sin puertas traseras:** no hay cuentas, contraseñas ni tokens embebidos en el código; no hay endpoints, parámetros ni modos *debug* que eleven privilegios. El rol solo se cambia desde la Consola de Firebase, con una cuenta de Google con 2FA, y queda registrado en los Cloud Audit Logs
RNF-SEG-03	Las soluciones, las pistas no reveladas y las explicaciones viven en `ejercicios/{id}/privado/clave`, que ningún estudiante puede leer. Solo las lee el autor o el servidor
RNF-SEG-04	Solo el servidor (Admin SDK) escribe en `sesiones`; las reglas niegan cualquier escritura del cliente
RNF-SEG-05	Cada función verifica el ID token con `verifyIdToken(token, true)`, que también rechaza los tokens revocados, y comprueba que la sesión pertenezca al `uid`
RNF-SEG-06	Validación de entrada en el servidor: cuerpo ≤ 64 KB, workspace ≤ 300 bloques y profundidad ≤ 50, tipos de bloque restringidos a la toolbox del ejercicio, identificadores con formato válido
RNF-SEG-07	Límite de frecuencia: como máximo 1 envío por segundo por sesión, controlado con la marca de tiempo del último envío guardada en la sesión
RNF-SEG-08	Todo texto escrito por un docente o un usuario se muestra con `textContent`, nunca con `innerHTML`. Política CSP en `vercel.json` sin `unsafe-inline` en scripts
RNF-SEG-09	La cuenta de servicio de Firebase solo existe en las variables de entorno de Vercel; los archivos `.env` y `*.json` de credenciales están en `.gitignore`
RNF-SEG-10	Contraseñas de al menos 8 caracteres con mayúscula, minúscula, número y símbolo (política de contraseñas de Firebase Auth) y protección contra la enumeración de correos activada
RNF-SEG-11	Solo HTTPS, con HSTS (Vercel)
RNF-SEG-12	Las dependencias de CDN se cargan con versión exacta
RNF-SEG-13	Las reglas se prueban con el emulador de Firestore: al menos un caso de *permitir* y uno de *denegar* por cada regla, incluido el intento de autoasignarse `rol = "docente"`

3.7 Requisitos de privacidad

ID	Requisito
RNF-PRI-01	Minimización: solo se recaban apodo, correo, contraseña (la gestiona Firebase y nunca se ve), resultados de quizzes y la aceptación del aviso. Ningún dato sensible en el sentido de la LFPDPPP

ID	Requisito
RNF-PRI-02	No se guarda la IP ni el código generado. Los workspaces enviados no se guardan: solo se registra el resultado del intento
RNF-PRI-03	No hay analítica de terceros, publicidad ni cookies de rastreo
RNF-PRI-04	Al eliminar una cuenta se borran su perfil, sus sesiones y su usuario de Authentication en una sola operación del servidor
RNF-PRI-05	El docente solo ve el apodo, nunca el correo, de los estudiantes

3.8 Confiabilidad, mantenibilidad y portabilidad

ID	Requisito
RNF-CON-01	Si falla una petición a `/api`, el workspace del estudiante no se pierde y se ofrece reintentar
RNF-CON-02	El DBA exporta Firestore a JSON con un script del Admin SDK al menos una vez por semana (el plan Spark no ofrece copias de seguridad gestionadas)
RNF-MAN-01	Canonicalización, puntaje y los 8 generadores tienen pruebas con `node:test`. Cada generador se prueba con un programa de referencia por tema
RNF-MAN-02	Añadir un bloque requiere tocar solo su definición y su función en cada generador; una prueba falla si a algún generador le falta el bloque
RNF-POR-01	Compatible con las 2 últimas versiones de Chrome, Firefox, Safari y Edge, en escritorio, Android e iOS

---

4. Verificación

Grupo	Método
RF de identidad	Prueba manual guiada por los criterios de aceptación (doc. 03) + pruebas de reglas en el emulador
RF de quiz y puntaje	Pruebas automáticas `node:test` (canonicalización, RN-01, RN-02) + prueba manual de extremo a extremo
RF de docente y analítica	Prueba manual con datos sembrados de alumnos ficticios
Usabilidad	Revisión con DevTools en 360, 768 y 1440 px; verificador de contraste; prueba con 3 usuarios
Rendimiento	Lighthouse (4G simulada); medición de tiempos de `/api` en los registros de Vercel
Seguridad	Emulador de reglas; revisión del código en busca de secretos (`git grep`); intento manual de escalar privilegios y de escribir sesiones
Privacidad	Inspección del modelo de datos frente a RNF-PRI; prueba de eliminación de cuenta

---

Anexo A. Catálogo de bloques del MVP

Categoría	Bloques
Programa	`programa` (raíz: equivale a `main`)
Variables	`declarar_variable` (tipo, nombre, valor), `asignar`, `variable`. Tipos: entero, decimal, texto, booleano, arreglo y objeto; los dos últimos se declaran con inferencia en los lenguajes tipados (`var`, `auto`, `val`)
Literales	`numero`, `texto`, `booleano`
Operadores	`aritmetica` (+ − × ÷ %), `comparacion` (= ≠ < ≤ > ≥), `logica` (y, o), `no`
Salida	`imprimir`
Control	`si`, `si_sino`, `mientras`, `para` (desde, hasta incluido, paso entero ≥ 1), `para_cada`, `romper`, `continuar`. "Sino si" se arma con un `si` dentro de "si no" y se genera como `elif` / `else if`; se descartó el mutador de Blockly por su complejidad de uso
Funciones	`definir_funcion` (nombre, tipo de retorno), `parametro` (tipo, nombre; se apilan en la ranura "parámetros"), `retornar`, `llamar_funcion` y `llamar_funcion_valor` (hasta 3 argumentos)
Arreglos	`crear_arreglo` (tipo de elemento, hasta 5 valores), `elemento_en`, `asignar_elemento`, `longitud`. Índices desde 0
Cadenas	`concatenar`, `longitud_texto`, `subcadena`, `mayusculas`, `minusculas`
POO (avanzado)	`definir_clase` (nombre, hereda de), `atributo`, `constructor`, `metodo`, `crear_objeto`, `obtener_atributo`, `asignar_atributo`, `llamar_metodo`, `llamar_metodo_valor`, `este`

Las conexiones están tipadas: las instrucciones solo encajan en cuerpos de instrucciones, los parámetros solo en la ranura "parámetros" y los atributos, constructores y métodos solo dentro de una clase. Definición en `js/bloques/definiciones.js`.

Tipos y su traducción:

Tipo	Java	Kotlin	C#	C++	C	Python	JS	PHP
entero	`int`	`Int`	`int`	`int`	`int`	—	—	—
decimal	`double`	`Double`	`double`	`double`	`double`	—	—	—
texto	`String`	`String`	`string`	`std::string`	`char*`	—	—	—
booleano	`boolean`	`Boolean`	`bool`	`bool`	`bool` (`stdbool.h`)	—	—	—

En Python, JavaScript y PHP el tipo no se escribe. En Kotlin se generan `val` o `var` según la variable se reasigne o no. Los tipos arreglo y objeto se declaran con inferencia (`var`, `auto`, `val`); en C++ los arreglos son `std::vector`.

Particularidades por lenguaje (definidas en `js/bloques/`):

- **Herencia:** los constructores no se heredan en Java, Kotlin, C# ni C++, así que una clase hija sin constructor reenvía el del padre (`super(...)`, `: base(...)`, `using Padre::Padre`). Las clases con hijas son `open` en Kotlin, y los métodos sobrescritos llevan `open` / `virtual` y `override`.

- **Cadenas:** Java compara con `.equals` y C con `strcmp`. En C, `imprimir(unir(...))` se genera como un solo `printf` con su formato; fuera de `imprimir`, las operaciones de texto usan funciones de apoyo que reservan memoria y lo indican en un comentario.
- **POO en C:** C no tiene clases (RN-10). Si se seleccionan bloques de POO, el código en C muestra un comentario que lo explica.

Anexo B. Algoritmo de comparación estructural

Entrada: el workspace serializado del estudiante y las soluciones aceptadas del ejercicio. **Salida:** `{correcto, diferencia?}`.

1. **Raíces.** Se toman los bloques de primer nivel. Los bloques sueltos que no sean `programa`, `definir_funcion` ni `definir_clase` se ignoran, y el cliente avisa de ellos antes de enviar.
2. **Canonizar** cada bloque de forma recursiva: `canon(b) = { t: tipo, f: campos, i: { entrada: canon(hijo) } con claves ordenadas, n: canon(siguiente) }`
   - Se descartan `id`, `x`, `y`, `collapsed`, comentarios y estado de UI.
   - En cada entrada se usa el bloque real si existe; si no, el bloque *shadow*.
   - Textos recortados; números como número (`3` = `3.0`).
3. **Renombrado de variables:** las variables, parámetros y atributos se sustituyen por `v0, v1…` en el orden de su primera aparición en un recorrido en preorden. Así `x = 5` y `edad = 5` son equivalentes. Los nombres de funciones, clases y métodos **no** se renombran, porque el enunciado los fija.
4. **Orden:** las definiciones (`funcion`, `clase`) se ordenan por nombre; el cuerpo de `programa` conserva su orden, porque el orden de las instrucciones sí importa.
5. **Comparar** la cadena JSON canónica del estudiante con la de cada solución. Si alguna coincide, la respuesta es correcta.
6. **Si no coincide:** se recorren en preorden el árbol del estudiante y el de la solución con el prefijo común más largo, y se informa el primer nodo distinto: su `id` original (para resaltarlo) y el tipo de diferencia (`bloque_distinto`, `valor_distinto`, `falta_bloque` o `sobra_bloque`).

Casos de prueba mínimos (`node:test`): soluciones idénticas; solo cambia la posición; variable renombrada; instrucciones en otro orden (incorrecto); valor distinto; bloque faltante; *shadow* sustituido por un bloque real; dos soluciones aceptadas.

Anexo C. Matriz de trazabilidad

RF	Caso de uso (doc. 02)	Historia (doc. 03)
RF-01	CU-01	HU-01
RF-02, RF-37, RF-38	CU-02	HU-02
RF-03	CU-02 (FA-1)	HU-03
RF-04, RF-05	CU-03	HU-04, HU-05
RF-06	CU-04	HU-04
RF-07, RF-08	CU-15	HU-20

RF	Caso de uso (doc. 02)	Historia (doc. 03)
RF-09	CU-14	HU-18
RF-10	CU-14 (FA-2)	HU-19
RF-11, RF-12, RF-13	CU-05	HU-06
RF-14, RF-15, RF-16, RF-26	CU-06	HU-07, HU-08
RF-17	CU-07	HU-09
RF-18 – RF-23	CU-08	HU-10, HU-11
RF-24	CU-09	HU-12
RF-25	CU-10	HU-13
RF-27, RF-28, RF-29, RF-31	CU-11	HU-14
RF-30	CU-12	HU-15
RF-32, RF-33	CU-13	HU-16
RF-34, RF-35	CU-13 (FA-1, FA-2)	HU-17
RF-36	CU-16	HU-21
RF-39	CU-01	HU-01